CS781 Capstone Project: In-House Legal Intelligence System

Rohit Batra
261110054
{userid}@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

A law firm that cannot send client documents to external services needs legal NLP it can host itself. We built a seven-phase pipeline under that constraint. Phase 1 pretrains four compact language models from scratch on the official corpus (103–139M parameters: decoder-only Transformer, encoder–decoder, stacked LSTM and dilated TCN) with our own 50K BPE tokenizer and an equal mix of acts, High Court, Supreme Court and tribunal text. Every later model starts from these weights and adapts them, with LORMA, LoRA, IA^3, prefix and prompt tuning, adapters, layer-wise fine-tuning and knowledge distillation, to entity recognition, rhetorical roles, judgment prediction, statute identification, summarisation and prior-case retrieval. The decoder transferred best to five of six tasks. Reading the right part of long documents, a probe-then-fine-tune schedule that ended repeated training collapses, and per-label calibration moved the scores most. Distilled students two-thirds the size kept their teachers' quality.

1 Competition Result

Codabench username: r_261110054 **Roll number:** 261110054

Table 1: Official phase scores and ranks (best valid submission per phase). Ranks recorded on 4 November 2026.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore F1 (0.8058)	133	4
2	Legal NER	Strict Macro-F1 (0.4912)	83	4
3	Rhetorical role prediction	Macro-F1 (0.5184)	47	7
4	Court judgment prediction	Macro-F1 (0.6795)	91	7
5	Legal statute identification	Macro-F1 (0.3037)	107	4
6	Abstractive summarization	BERTScore F1 (0.8354)	77	8
7	Prior Case Retrieval (PCR)	Micro-F1@7 (0.2534)	0*	4

*The Phase-7 leaderboard was still open when ranks were recorded.

Total valid Codabench submissions across all phases: 38 **Date ranks were recorded:** 4 November 2026

We met the minimum of four distinct submissions in every phase, covering all four backbones (Phase 7: three and a distilled student); Table 6 lists all 38. **GitHub:** https://github.com/Rohit2004Batra/cs781-legal-lm-capstone

2 Introduction and Problem Description

Client confidentiality rules out sending judgments and pleadings to a hosted language model, so a law firm needs models it can train and run on its own hardware, built here from only the official `Exploration-Lab/CS-781-Capstone` data.

Phase 1 pretrains a language model of 50M–300M parameters from scratch. Its test asks for exactly 128-word continuations of 1,000 legal prompts of 512 words, scored by BERTScore F1 [1] with ROUGE-L [2] and BLEU [3] as secondary metrics. Phases 2–7 adapt that model to six tasks. Phase 2 (legal NER) returns character spans `[start, end, label]` over 12 entity types, scored by strict Macro-F1. Phase 3 assigns one of 13 rhetorical roles to every provided sentence of Corporate Law (CL) and IT judgments, scored by Macro-F1. Phase 4 predicts a binary outcome per case (Macro-F1). Phase 5 returns the set of relevant statutes out of 100 per case, scored by Macro-F1 over statutes. Phase 6 writes a headnote-style summary (BERTScore F1), and Phase 7 returns exactly seven candidate case IDs from the pool for each query (Micro-F1@7).

Constraints shaped the design: every model starts from our own Phase-1 weights (no other pretrained model, external corpus or LLM API); each phase needs four distinct submissions, including LORMA [4] and knowledge distillation; compute was free Kaggle T4 GPUs in 12-hour sessions; strict span matching makes exact offsets essential; statutes are multi-label with a long tail; and BERTScore rewards fluent, on-topic summaries rather than exact wording.

We investigated five questions: whether four architectures with very different inductive biases, trained on the same budget, transfer differently; which parameter-efficient technique suits which backbone; how to represent documents that run to tens of thousands of tokens; how much decision calibration (thresholds, span caps, decoding) matters relative to training; and whether a smaller distilled student keeps its teacher's quality. We built everything end to end, from tokenizer to submission files. We deliberately did not pipeline phases into one another, so each remains a clean test of transfer from Phase 1, and we submitted no lexical or hybrid ranker in Phase 7.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

We used only `Exploration-Lab/CS-781-Capstone` with its official phase folders, downloaded inside each notebook. Table 2 gives the size of every split and the finding from each phase's analysis notebook that changed our design. Phase 2 has two labelled folds and Phase 6 no dev set, so we held out 14 of 70 NER and 150 of 7,030 summarisation documents; Phase 3 has separate CL and IT files.

3.2 Corpus statistics that actually informed design

Phase 1. Under our tokenizer the training split holds 2,327M High Court, 1,607M tribunal, 252M Supreme Court and 150M act tokens (53.7%, 37.1%, 5.8% and 3.5%; 4.34B in total, with 83.6M held out). A uniform sample would be dominated by High Court style, so each training window first draws a source with equal probability (25% each), upsampling acts about 7× so the model also learns statutory drafting style.

Phases 2–6. NER labels are skewed (DATE 1,575 spans, R.COUNSEL 184), and spans have a median of 13 characters and reach 233, precedents being longest. Rhetorical roles follow a document order (facts, arguments, precedents, ratio, ruling); CL dev has far more argument sentences than IT (41% vs 12%), IT three times CL's share of Statute sentences, and Issue and Dissent are very

Table 2: Data per phase and the analysis result that shaped the design.

Ph.	Train / dev / test	Finding → decision
1	4.34B tokens, 4 sources; 2% held out	High Courts and tribunals are 91% of tokens (acts 3.5%) → sample the four sources equally.
2	70 docs (2 folds) / – / 35; 8,393 spans	Only 3.8% of gold starts lie on a BPE token start (byte-level BPE attaches the leading space) → predict boundaries per character.
3	80 / 10 / 10 docs (CL + IT); 17k sentences	Median sentence 32 tokens but documents up to 70k; Issue is 0.2% of CL; Dissent appears only in CL train → sentence encoder + document BiLSTM, class weights.
4	31.6k unique / 3,505 / 1,517	**All test cases also occur in dev, with labels** → never train or tune on them ("dev-clean", 1,988 cases). Outcome words differ by label only near the end → read the last chunks.
5	42.8k / 10.2k / 13.0k; 100 IDs	3.7 statutes per case; 42% of dev label sets never occur in train → independent sigmoids, per-statute thresholds.
6	7,030 / – / 100	Summaries are 22% of document length and draw on the whole case; 4% novel words → summarise every chunk and join.
7	827 / 118 / 237 queries; pools 4,320 / 1,023 / 1,727	Each query is in its own pool; BM25 scores 0.13 on full text but 0.33 on the first 2,000 words → exclude the query; read first and last chunks.

Table 3: Phase-7 prior-case retrieval data (relevant cases per query, and dev reference rankers that were never submitted).

Split	Queries	Pool	Relevant/query (median / mean / max)	Oracle Micro-F1@7
Train	827	4,320	4 / 6.4 / 52	0.639
Dev	118	1,023	5 / 8.3 / 92	0.618
Test	237	1,727	–	–
Dev Micro-F1@7: BM25 full text 0.128; BM25 first 2,000 words 0.326; TF-IDF (sublinear) 0.368.				

rare. In Phase 4, train is 59/41 while dev and test are balanced, and "set aside" appears in the last 1,000 characters of 25% of label-1 and 6.6% of label-0 cases but equally often in the first 1,000: the outcome-bearing text sits at the end even after the ruling is removed. Statute frequencies span 43× (272 to 11,730 training cases) and pairs co-occur strongly ($P(28 \mid 20) = 0.93$), yet 42% of dev label sets are unseen, so whole label sets cannot be predicted. Phase-6 summaries compress the document to 22%, draw evenly on the whole case and add only 4% novel words; extractive baselines already reach ROUGE-1 0.52.

Phase 7. Table 3 summarises the retrieval data. Queries and candidates have similar lengths (median 4.4k–5.8k tokens, maximum 507k), so one encoding policy fits both; every query is in its own pool but never relevant to itself. Because exactly seven answers are returned while gold sets range from 1 to 92, the best possible Micro-F1@7 on dev is only 0.62. Citations are masked, removing the most direct cue.

3.3 Preprocessing

Text was kept cased (38K of the 50K tokens contain capitals), except in Phase 4, whose dev and test splits are lower-case. Citations and section symbols are left to the byte-level tokenizer, which has no unknown tokens. Masked placeholders cost 3–5 BPE tokens each, so we replaced them with one plain word (`<SECTION>`→"Section", `<CITATION>`→"Citation", `<ORG>`→"Org") or removed them where they also hide ordinary words (`<ENTITY>` in Phase 5).

For Phase 2 we never normalised whitespace or Unicode. Each 512-token window keeps a character-to-token map built from the tokenizer's offsets; predictions are made per character and written back as the original offsets. As a check, decoding the gold boundaries through this path

reproduces all 1,861 validation spans exactly. Phase 3 used the provided sentence lists unchanged.

Long documents are chunked rather than truncated. Phases 4, 5 and 7 split each case into chunks of 510 tokens and select chunks per task: the last two (Phase 4), up to 8 (16 at test) chosen by evidence (Phase 5), and the first 8 and last 8 (Phase 7). Phase 6 uses sentence-aligned chunks of 1,200 tokens. In Phase 7, queries and candidates share one encoder and one chunking policy.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

We first trained our own byte-level BPE tokenizer (vocabulary 50,000, minimum frequency 2, 25M characters from each source; special tokens `[PAD]` `[UNK]` `[BOS]` `[EOS]`), so that frequent legal words such as "Section", "appellant" and "Tribunal" are single tokens and citations decompose into stable pieces.

We then designed four architectures. A is a pre-LayerNorm decoder-only Transformer [5] with 12 layers, hidden size 768, 12 heads, a GELU feed-forward layer of 3,072, learned positional embeddings, a context of 2,048 and tied input/output embeddings (125.0M parameters). B is an encoder–decoder with 6+6 layers of the same width and a shared embedding, trained to predict the next 1,024 tokens from the previous 1,024 (139.3M). C is a three-layer LSTM [6] with 1,280 units (103.4M). D is a TCN [7] of 10 residual blocks, each with two weight-normalised causal convolutions, dilations doubling from 1 to 512 and a receptive field of 4,093 tokens (114.2M). Each notebook counts parameters with `sum(p.numel())` and asserts the count lies in 50M–300M.

We chose four families on purpose, each with a different inductive bias under the same budget. The decoder matches the Phase-1 task and attends over its whole context; the encoder–decoder reads the prompt bidirectionally, and its encoder suits classification. The LSTM runs in linear time with constant memory over documents of any length, and its step-by-step state suits local decisions such as entity boundaries. The TCN is fully parallel but has a fixed, very wide receptive field without attention cost. Comparing attention, recurrence and convolution shows which bias transfers best to each task.

All four were trained with next-token cross-entropy on 100M tokens (2.3% of the corpus, the most the Kaggle sessions allowed), drawn with the equal source mix above. We used AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1), a peak learning rate of 3×10^{-4} with warm-up and cosine decay to 3×10^{-5}, bf16, and 16K tokens per optimiser step (8K for C, which used 1,024-token windows), and selected checkpoints on the held-out loss.

At submission time, decoding produced exactly 128 words by generating extra tokens and cutting at the 128th word. We used near-greedy sampling for A and C (temperature 0.1, top-p 0.95), temperature 0.7 with top-p 0.95, top-k 50 and repetition penalty 1.1 for B, and temperature 0.8 for D. The near-greedy setting was our first, untuned choice and in hindsight a mistake: A had by far the lowest perplexity (20.3, against 31.8, 48.0 and 48.2) but the lowest BERTScore (0.793, against 0.806, 0.802 and 0.805) because it fell into repetitive loops.

4.2 Transfer protocol (Phases 2–7)

Every downstream model initialises its backbone from a Phase-1 checkpoint and adds a task head (Figure 1); no task model was trained from scratch. Across the phases we implemented full fine-tuning (P2), layer-wise fine-tuning of the top layer (P4, P5, P7), BitFit [8] (P4), LoRA [9] (P2, P3, P5, P6), IA^3 [10] (P3), prefix tuning [11] (P3), prompt tuning [12] (P3, P7), Houlsby and bottleneck adapters [13] (P4, P5, P6), knowledge distillation [14] in every phase, and LORMA [4]

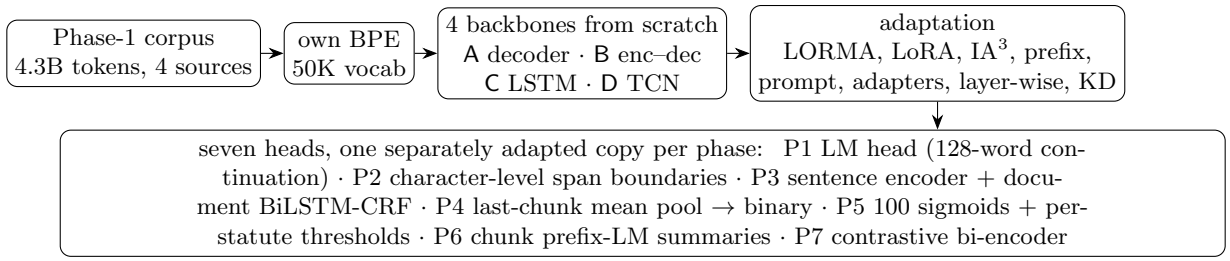


Figure 1: System overview: one set of Phase-1 backbones and seven task heads, each phase a separately adapted descendant.

in every phase. LORMA replaces each frozen weight W_0 with a multiplicative update

$$W_{\text{eff}} = \Big(I + \tfrac{\alpha}{r} BA\Big) W_0, \qquad r = 8,\ \alpha = 16,\ B = 0 \text{ at the start}, \tag{1}$$

applied to the q/k/v/out/fc1/fc2 projections of A and to the recurrent gate matrices of C.

LORMA was the best technique in Phases 2 and 4–7 (Table 6). Full fine-tuning of A on 56 NER documents over-fitted (0.19), and prefix tuning lost to LoRA by 0.05 in Phase 3. Layer-wise fine-tuning of C was solid in classification but over-fitted in retrieval, adapters and prompt tuning on D were mid-table, and IA^3 and BitFit on B's encoder were weakest, matching its poor frozen features.

Two pieces of infrastructure carry across phases. First, A's residual stream has a few nearly constant dimensions ($|x| \approx 46$ against a mean of 2) that dominate pooled vectors, so chunk features are *standardised per dimension* with statistics from the frozen model. Second, from Phase 4 on we used *LP-FT* [15]: a linear probe (or retrieval projection) is fit on cached frozen features and copied into the model as "epoch 0", then the adapter and head are fine-tuned at a learning rate of 10^{-4}, and the best of epochs $0, \ldots, E$ on validation is kept.

4.3 Phase 2 - Legal NER

Since almost no gold start lies on a token start, token classification followed by offset reconstruction cannot be exact, so we built a character-level span-extraction head. Each character receives its token's backbone state plus a 32-dimensional character embedding; a small convolution follows, and two heads give start and end probabilities for each of the 12 labels, so offsets are predicted directly in character space. Training uses binary cross-entropy with a positive weight capped at 30, because boundaries are only 0.01–0.08% of characters, and output biases initialised from label prevalence. Every character is scored, so documents with many spans need no special handling; 512-token windows overlap with stride 384 and are averaged in character space so no entity is cut at a window edge.

Decoding pairs each start above its label's threshold with the best-scoring end within a per-label maximum length, then resolves overlaps by keeping the higher-scoring span; thresholds and caps were tuned on validation. The best system, C with LORMA on the recurrent weights, trains 0.79M parameters. An earlier version applied an additive adapter to the hidden states, which is not LORMA; it scored 0.372 against 0.392 for true LORMA before calibration.

4.4 Phase 3 - Rhetorical role prediction

Each sentence is encoded on its own (at most 128 tokens) into [mean; final state], projected to 512 dimensions and combined with a domain embedding and its relative position. A two-layer document BiLSTM then gives every sentence left and right context across a window of 128 sentences, and from

S7 on a linear-chain CRF [16] adds learned transition constraints and decodes the role sequence with Viterbi. CL and IT were trained jointly, because 40 documents per domain is too few for separate models, and the domain embedding lets the model shift its priors. The loss is cross-entropy with class weights $(N/Kn_c)^{0.5}$, capped at 10.

The best submission (S6) is a KD student. It keeps layers $0, 2, \ldots, 10$ of A (DistilBERT-style [17]) with LoRA, is trained with CRF NLL, weighted cross-entropy and $T^2 \cdot \mathrm{KL}$ ($T = 2$) against the S7 teacher, and is retrained on train+dev for the epoch count chosen on dev.

4.5 Phase 4 - Court judgment prediction

Each case is represented by its last two chunks of 510 tokens. A chunk vector [mean; final state] is standardised, the chunk vectors are averaged, and a linear layer gives the two-class output. Attention pooling was dropped during the collapse diagnosis (Section 7); Phase-2/3 outputs were not used. Macro-F1 rather than accuracy is the right target: a majority-class predictor would score about 0.33 Macro-F1 on the balanced test set, exactly what our collapsed runs produced. Class-weighted cross-entropy and a decision threshold tuned on dev-clean (0.41) handle the shift from the 59/41 training balance.

4.6 Phase 5 - Legal statute identification

Up to 8 chunks are selected: the first, the last, and those with the most "Section" or "Act" tokens. Each case is pooled as [mean; max] over chunks and passed to 100 independent sigmoids, trained with binary cross-entropy whose positive weight is $\sqrt{\mathrm{neg}/\mathrm{pos}}$, capped at 10. Every statute has at least 68 dev positives, so each gets its own threshold, reported cross-fitted (tuned on half of dev, scored on the other half); every case receives at least one statute. Rare statutes are handled by the capped positive weight and per-statute thresholds rather than resampling.

4.7 Phase 6 - Abstractive summarization

We chose decoder-only continuation because A was the stronger backbone (B with LoRA was submitted for comparison), and chunk-then-merge because only 19% of documents fit in 2,048 tokens. Documents are cut into sentence-aligned chunks of 1,200 tokens, and each reference summary sentence is assigned to the chunk containing its best-matching source sentence (89% match with word recall ≥ 0.5), giving each chunk its own target of at most 640 tokens. A with LORMA is trained as a prefix language model on `[BOS] chunk ‘‘Summary:’’ target [EOS]`. At inference every chunk is summarised with a KV cache and 4-gram blocking, and the chunk summaries are joined with duplicate sentences removed.

The original decoding was short and repetitive (20–39% repeated trigrams). A grid on 60 validation documents chose a minimum length of 22% of each chunk's length and a CTRL repetition penalty [18] of 1.2. The tuned runs replaced the originals; no post-training or alignment stage was used.

4.8 Phase 7 - Prior Case Retrieval (PCR)

We built a bi-encoder in the style of dense retrieval [19], with one shared encoder for queries and candidates. Each chunk's [mean; final state] features are standardised and projected to 256 dimensions, and the case vector is the normalised mean over the first 8 and last 8 chunks of 510 tokens, which bounds the cost of documents up to 507k tokens. Candidates are ranked by cosine similarity, with the query removed from its own ranking. Training uses InfoNCE [20] at temperature

Table 4: Hyperparameters of the final model in each phase (total parameters; trainable in brackets).

Hyperparam.	P1 B	P2 C+LORMA	P3 KD student	P4 A+LORMA	P5 A+LORMA	P6 A+LORMA	P7 A+LORMA
Parameters (M)	139.3	104.2 (0.79)	85.5 (3.04)	126.4 (1.4)	126.7 (1.63)	126.4 (1.33)	126.7 (1.72)
Context length	1k→1k	512/384	128/sent.	2×510	≤8×510	1,200 + 640	16×510
Peak LR	3e-4	2e-4	3e-4/1e-3	1e-4	1e-4	2e-4	1e-4
Batch×acc.	1×8	2×1	128 sent.	8 cases	16 ch.×2	4×4	4 pairs
Epochs / steps	6,104 steps	3 (best 2)	20 (best 17)	4 (best 3)	3	2	3 (best 2)
Dropout	0.1	0	0.2 head	0.1 head	0.1 head	0	0
Decode / thr.	T0.7, rep 1.1	thr. + caps	Viterbi	thr. 0.41	per-statute	22%, rep 1.2	top-7 cosine

0.05 over one relevant case, two hard negatives from the TF-IDF top 50, and all other documents in the batch; LP-FT first fits the projection on frozen features. BM25 [21] and TF-IDF served only as references and negative miners, never in a submitted ranking. The KD student (S4) keeps 6 of A's 12 layers and matches the teacher's similarity distribution over each batch with $T^2 \cdot \mathrm{KL}$ at $T = 2$, plus one teacher-mined hard negative per pair.

Several approaches failed: max pooling and first-chunk-only encoding lost 0.09 and 0.16 (Table 5); layer-wise fine-tuning of C over-fitted (dev 0.217, test 0.180); fine-tuning added only 0.02–0.03 over the projection with about 15 in-batch negatives; and no neural ranker beat TF-IDF on dev (0.368).

5 Experiments and Implementation Details

5.1 Compute and training protocol

All 46 training notebooks ran on Kaggle T4s (two with DataParallel after Phase 1), 30–160 minutes per downstream run. Runs are killed at session limits, so every notebook saves `latest.pt` every few hundred steps and at each epoch, together with the optimiser state, scheduler state and sampler position, so an interrupted run resumes exactly.

Downstream runs used AdamW with weight decay 0.01, a linear warm-up over 3–6% of steps followed by linear decay, gradient clipping at 1.0 and fp16 autocast; batch sizes, dropout and learning rates are in Table 4. The frozen backbone runs without dropout, which mattered in Phase 4. Checkpoint selection and early stopping tracked the task metric on validation (span-F1, Macro-F1, Micro-F1@7), never the loss, except in Phase 6, where validation loss chose the epoch and ROUGE chose the decoding settings. Phase 1 selected on held-out loss (perplexity).

5.2 Local evaluation

Every phase has a labelled held-out set built only from public train/dev data: 2% of Phase-1 documents held out by hash, 14 of the 70 labelled NER documents, the official dev sets in Phases 3, 5 and 7, the dev set minus the 1,517 test cases in Phase 4 (dev-clean), and 150 training documents in Phase 6. Each notebook implements the official metric and validates `prediction.json` before upload: all and only the test IDs, exact types, sentence counts (Phase 3), unique statute IDs with at least one per case (Phase 5), and seven unique pool IDs excluding the query (Phase 7).

Local and Codabench scores mostly agree (Table 6); the cross-fitted Phase-5 estimates were within ±0.01 of test for all four runs. Our first Phase-3 score averaged Macro-F1 only over labels present (0.476 vs Codabench's 13-label 0.402), so we switched to the 13-label average. The other

Table 5: Controlled ablations on local validation (Phase 6 in ROUGE-1; Phase-3 prefix row on test).

Ph.	Change (all else fixed)	Δ	Ph.	Change (all else fixed)	Δ
2	per-label span caps + thresholds	+0.102	5	first chunk only vs evidence chunks	−0.035
3	document BiLSTM vs sentence-only	+0.158	5	thresholds scored on tuning data	+0.015 bias
3	Viterbi CRF vs argmax	+0.042	6	all chunks joined vs first chunk	+0.158
3	prefix tuning vs LoRA	−0.049	6	tuned decoding, same weights	+0.060
4	last 2 vs first 2 chunks	+0.102	7	contrastive projection on frozen C	+0.074
4	LORMA fine-tuning over its probe	+0.088	7	mean vs max pooling over chunks	+0.089
4	4 epochs vs 2 (same recipe)	+0.024	7	first 8+last 8 vs first chunk	+0.157

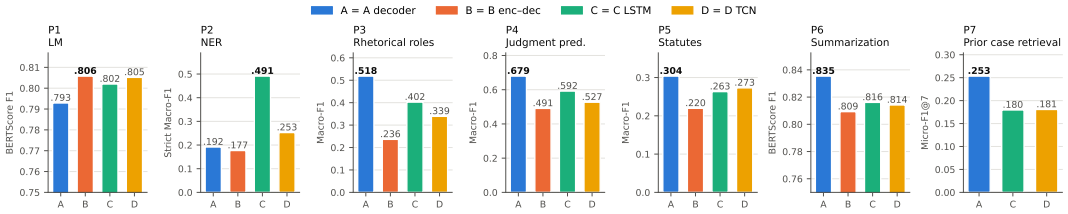


Figure 2: Best official score per backbone and phase (KD students under their own backbone; Phase 6 with tuned decoding; no B run in Phase 7; BERTScore axes start at 0.75). A wins Phases 3–7; C wins NER.

gaps come from small validation sets: Phase 3 has only 10 dev documents, and in Phase 7 every run lost 0.02–0.04 on test because choosing the epoch and chunk policy on 118 queries over-fits slightly.

5.3 Ablations and controlled comparisons

Table 5 changes one factor at a time, either against the same trained model or between two runs that differ only in that factor. Document structure and decision calibration produced the largest effects, larger than switching adapters on a fixed backbone. Two ablations are negative: prefix tuning loses to LoRA under an identical pipeline, and max pooling loses badly to mean pooling for retrieval.

6 Results

Table 6 lists every submission with its local validation score and its official Codabench primary and secondary metrics; Table 1 gives the best per phase. Figure 2 summarises the best score of each backbone in each phase.

Which system won, and why. The decoder A adapted with LORMA (or distilled from it) won five of six downstream phases. Frozen-feature probes predicted this ranking before any fine-tuning: on Phase-3 sentence-only probes A scored 0.283, C 0.235, D 0.226 and B's encoder 0.117, and in frozen Phase-7 retrieval the order was A 0.238, D 0.122, C 0.112. The explanation is representational: A had the longest pretraining context and lowest perplexity, so its states encode more of the document, and LORMA reshapes every projection slightly while touching about 1% of the parameters. NER is the exception. A boundary decision is local and character-precise, and the LSTM's state at a character, adjusted multiplicatively on the recurrent gates, gave the sharpest boundaries. In Phase 1 the winner (B) was decided by decoding, not model quality.

Learning curves. Figure 3 shows the task metric against epoch for the LP-FT runs. The probe (epoch 0) already recovers most of the attainable score, and fine-tuning adds 0.02–0.09. Phase-1 loss histories were kept inside the checkpoints but not exported; the selected points are A

Table 6: Per-phase variant comparison, all 38 submissions. Local: held-out score in the primary metric (Phase 3: 13-label dev Macro-F1). †: not counted as final. Best counted run per phase in bold.

Phase	Variant	Local	Codabench primary	Codabench secondary
1	A decoder, from scratch	PPL 20.30	0.7929	R-L 0.173, BLEU-4 0.021
1	**B encoder–decoder, from scratch**	PPL 31.82	**0.8058**	R-L 0.162, BLEU-4 0.015
1	C stacked LSTM, from scratch	PPL 47.97	0.8020	R-L 0.175, BLEU-4 0.017
1	D TCN, from scratch	PPL 48.17	0.8052	R-L 0.185, BLEU-4 0.021
2	S1 A full fine-tuning	0.225	0.1916	P 0.196, R 0.209
2	S2 B encoder + LoRA	0.198	0.1768	P 0.211, R 0.194
2	**S3 C + LORMA**	0.494	**0.4912**	P 0.469, R 0.571
2	S4 KD D → 6-block D	0.220	0.2527	P 0.281, R 0.264
3	S1 C + LORMA + doc BiLSTM	0.40	0.4024	P 0.412, R 0.416
3	S2 A + LoRA + doc BiLSTM†	0.440	0.4825	P 0.535, R 0.540
3	S3 B encoder + IA^3	0.219	0.2364	P 0.308, R 0.252
3	S4 D + prompt tuning	0.329	0.3391	P 0.363, R 0.347
3	S5 A + prefix tuning + CRF†	0.425	0.4527	P 0.499, R 0.467
3	S7 A + LoRA + BiLSTM-CRF	0.450	0.5019	P 0.537, R 0.494
3	**S6 KD S7 → 6-layer A + LoRA**	0.458	**0.5184**	P 0.582, R 0.502
4	S1 A + LORMA (random head, collapsed)†	0.345	0.3475	P 0.503, R 0.988
4	S1 A + LORMA LP-FT, 2 epochs†	0.671	0.6732	P 0.710, R 0.597
4	**S1 A + LORMA LP-FT, 4 epochs**	0.695	**0.6795**	P 0.662, R 0.744
4	S2 C layer-wise FT	0.562	0.5615	P 0.574, R 0.503
4	S3 B encoder + BitFit	0.531	0.4908	P 0.502, R 0.634
4	S4 D + bottleneck adapters	0.578	0.5269	P 0.534, R 0.478
4	S5 KD S2 → 2-layer C	0.613	0.5920	P 0.603, R 0.552
5	**S1 A + LORMA LP-FT**	0.298	**0.3037**	Micro-F1 0.399
5	S2 C layer-wise FT	0.273	0.2632	Micro-F1 0.369
5	S3 D + bottleneck adapters	0.271	0.2730	Micro-F1 0.376
5	S4 KD ensemble → B encoder + LoRA	0.215	0.2196	Micro-F1 0.314
6	**S1 A + LORMA** (original† → tuned decoding)	R-1 0.521	0.8287 → **0.8354**	R-1 0.527, R-L 0.240
6	S2 B + LoRA (original† → tuned)	–	0.8033 → 0.8093	R-1 0.373, R-L 0.200
6	S3 D + Houlsby adapters (original† → tuned)	–	0.8060 → 0.8142	R-1 0.393, R-L 0.202
6	S4 KD A → C (original† → tuned)	–	0.8086 → 0.8161	R-1 0.435, R-L 0.198
7	**S1 A + LORMA bi-encoder**	0.269	**0.2534**	P@7 0.259, R@7 0.248
7	S2 C layer-wise FT bi-encoder	0.217	0.1796	P@7 0.183, R@7 0.176
7	S3 D + prompt tuning bi-encoder	0.198	0.1807	P@7 0.184, R@7 0.177
7	S4 KD S1 → 6-layer A	0.280	0.2374	P@7 0.242, R@7 0.233

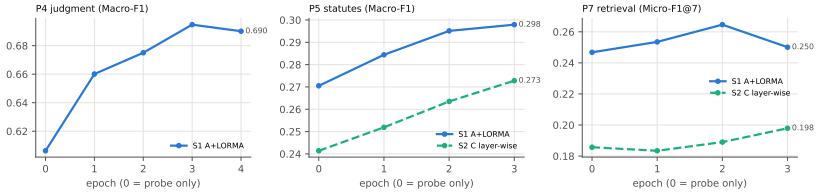


Figure 3: Validation metric vs epoch for the LP-FT runs; epoch 0 is the probe or projection alone.

reached validation loss 3.01 (perplexity 20.3) at step 4,500 of 6,104, B 3.46 (31.8) at step 6,000 of 6,104, C 3.87 (48.0) at step 12,000 of 12,208, and D 3.87 (48.2) at its last step.

Per-class results. DATE (0.91) and COURT (0.77) are reliable in NER while APP, AUTH, RESP, WIT and PREC stay at 0.25–0.30; per-statute F1 rises from 0.20 in the rarest quartile to 0.45 in the most frequent.

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

Phases 1–3. Near-greedy A fell into repetitive loops, and all four models' continuations drifted from the prompt's topic after a few sentences, compressing BERTScore to a 0.013 range. In NER, the uncalibrated S3 model predicted spans up to 512 characters (gold spans are under 150) until per-label caps removed this; AUTH dominates the false positives (436 false vs 105 true), and APP

vs RESP hinges on a position relative to "versus" often far from the name. Role errors concentrate where the role depends on the court's later use of a sentence or on who is speaking, and on rare roles.

Phase 4. A with LORMA collapsed to a constant prediction in four runs (Macro-F1 0.35). Removing attention pooling, outlier dimensions and backbone dropout did not help; the cause was a random head combined with multiplicative updates, which distort the features before the head finds the weak signal. LP-FT fixed it (probe 0.606, fine-tuned 0.695). Remaining errors favour acceptance (recall 0.744, precision 0.662): with the ruling removed, partly allowed or remanded cases read like acceptances.

Phase 5. The dominant error swaps a specific statute for a frequent one (12–14 missed for 5; 34, 61, 77 missed for 97): with references masked, the model falls back on what is typical.

Phase 6. Independently written chunk summaries restate facts and sometimes split the holding; longer tuned outputs raised ROUGE-1 and BERTScore but lowered ROUGE-L for three models.

Phase 7. P@7 is 0.05–0.08 for single-answer queries and R@7 0.09–0.13 for queries with 16+ answers; S1 finds nothing for 18 of 118 dev queries. All rankers stay below TF-IDF, but S1 and TF-IDF together find 393 relevant dev cases against 331 for TF-IDF alone.

7.2 What the metrics do not measure

BERTScore and ROUGE reward similarity to one reference, so a fluent summary that misstates the holding can still score well. Strict span-F1 gives no credit for an entity one character off, and Micro-F1@7 caps any system at 0.62 on dev. The scores rank systems; they do not certify them for a firm.

7.3 Limitations and what you would do next

Each Phase-1 model saw 2.3% of the corpus, fine-tuning was held to 2–4 epochs and contexts to 2,048 tokens, and there was no human legal evaluation. Next we would pretrain A on 1B tokens with a 4,096 context, add contrastive pretraining on citing–cited training pairs, raise in-batch negatives with gradient caching, and mix neural and lexical retrieval scores.

7.4 Surprising findings

The best-perplexity model had the worst Phase-1 BERTScore yet became the best downstream backbone. KD students beat their teachers in Phase 3 (0.518 vs 0.502 at 66% of the parameters) and Phase 4 (0.592 vs 0.562), and in Phase 7 on dev only (0.280 vs 0.269, 1.99× faster). In Phase 6, two decoding numbers beat any architectural change.

8 Conclusion

We delivered a self-hosted legal NLP stack that runs on one 16 GB GPU. Reading the right part of each document, stable optimisation (standardisation, LP-FT) and calibrated decisions moved the metrics; the choice of adapter mattered much less. Under a fixed budget, backbone quality, visible in frozen probes, decides most of the outcome. A skeptical partner should still not trust unreviewed summaries, cue-based judgment predictions, or a retriever below word matching.

LLM Usage Declaration

Completing this section is mandatory. Inconsistent answers relative to the code and the prose will be treated as an integrity issue.

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: Claude (Anthropic), used from Phase 3 onwards, for the repository documentation and for this report. No LLM was used for Phases 1 and 2, which I designed and implemented myself with the help of the lecture slides.

Q2. For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Explaining LoRA, LORMA, LP-FT, KD and retrieval losses
Ideation / brainstorming at a high level	Yes	Per-phase plans and technique choices
Debugging small ancillary snippets	Yes	Diagnosing errors from notebook outputs
Plotting / visualization code	Yes	Wrote `report/make_figures.py`
Grammar / writing style of the report	Yes	Drafting and editing the report text
Report structure / section wording	Yes	Report and README structure
Designing the model architecture	Yes	Phase 3–7 task heads and adaptation setups (Phase 1–2: no)
Hyperparameter search / tuning	Yes	Settings and the Phase-6 decoding grid
Analysis of results / conclusions	Yes	Drafting the README and report analysis
Writing the core training / inference code	Yes	Phase 3–7 notebooks (Phase 1–2: no)
Other (specify)	Yes	Data-analysis notebooks, repository documentation

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer: Yes. The papers in the bibliography (Transformer, LSTM, TCN, LoRA, LORMA, LP-FT, IA^3, prefix and prompt tuning, adapters, BitFit, knowledge distillation, DistilBERT, CTRL, CRF, InfoNCE, DPR, BM25, and the BERTScore, ROUGE and BLEU metrics), the capstone document, the official dataset and its starter notebook, and the Codabench evaluation output.

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name

at least two things you tried that failed and what you concluded from them.

[Write 8–12 sentences here in your own words.]

References

[1] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," *International Conference on Learning Representations*, 2020.

[2] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, pp. 74–81, 2004.

[3] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in *Proceedings of ACL*, pp. 311–318, 2002.

[4] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[7] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[8] E. Ben Zaken, S. Ravfogel, and Y. Goldberg, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proceedings of ACL*, 2022.

[9] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[10] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Advances in Neural Information Processing Systems*, 2022.

[11] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proceedings of ACL-IJCNLP*, 2021.

[12] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proceedings of EMNLP*, 2021.

[13] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning*, 2019.

[14] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[15] A. Kumar, A. Raghunathan, R. Jones, T. Ma, and P. Liang, "Fine-tuning can distort pretrained features and underperform out-of-distribution," in *International Conference on Learning Representations*, 2022.

[16] J. Lafferty, A. McCallum, and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in *International Conference on Machine Learning*, 2001.

[17] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[18] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," *arXiv preprint arXiv:1909.05858*, 2019.

[19] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of EMNLP*, 2020.

[20] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[21] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.